A

Project Based Learning Report

For

"Handwritten Digit Recognition"

is submitted in partial fulfillment of the requirement for the award of degree of

Bachelor of Technology

(VI Semester B. Tech. for the course **Basics of Machine & Deep Learning** (**PCCAD602T**))

in

Artificial Intelligence & Data Science

Submitted by

Divyanshi R. Bokade (AD22054) **Vaibhav D. Charde (AD22055)**

Samarth S. Sharma (AD2256) **Peeyush S. Kaware (AD22057)**

Under the guidance of

Prof. Kalyani Pendke

Assistant Professor

Department of Emerging Technologies

(Artificial Intelligence & Data Science)

S. B. Jain Institute of Technology, Management and Research, Nagpur

(An Autonomous Institute Affiliated to R. T.M. Nagpur University)

Academic Session: 2024-2025 (EVEN)

Problem Statement: Handwritten Digit Recognition

Develop a robust CNN and ResNet based model for handwritten digit recognition (0-9) to address challenges like variations in handwriting styles, sizes, and orientations. Using the MNIST dataset that leverage the feature extraction capabilities of CNNs and ResNet to ensure high accuracy and computational efficiency.

Objectives:

- **Develop a Reliable System**: Create a robust solution for handwritten digit classification (0–9) while addressing noise and handwriting variations.
- **Leverage ResNet**: Use ResNet's residual learning for deeper networks, improving precision in complex handwriting and noisy data.
- **Evaluate Models**: Compare CNNs and ResNet on accuracy, efficiency, and robustness.
- **Ensure Generalization**: Train models on diverse datasets to adapt to various handwriting styles and orientations.
- **Achieve Accuracy**: Utilize CNNs and ResNet for high-precision digit recognition with efficient feature extraction.

Introduction:

Handwritten digit recognition is a fundamental challenge in **computer vision** and **machine learning**, aiming to accurately classify digits (0–9) from handwritten images. This task is complex due to the **variations in handwriting styles, sizes, orientations, and noise** in the images. However, advancements in **deep learning** have significantly improved recognition accuracy, making it an essential tool for **automated data processing**.

To tackle these challenges, this project focuses on developing a **robust handwritten digit recognition model** using **Convolutional Neural Networks (CNNs) and Residual Networks (ResNet)**.

Importance of Handwritten Digit Recognition

- Widely used in **postal services, banking (cheque processing), and document digitization**.
- Plays a crucial role in **automated form processing and AI-driven applications**.
- Serves as a benchmark for evaluating machine learning and deep learning models.

Challenges in Handwritten Digit Recognition

- **Diverse handwriting styles** → Different individuals write in unique ways.
- **Size and orientation variations** → Digits can be skewed, rotated, or stretched.
- **Noisy and low-quality images** → Blurred or distorted inputs affect accuracy.
- **Data complexity** → Digits like '3' and '8' or '1' and '7' can be difficult to distinguish.

Deep Learning Approaches: CNNs and ResNet

a) Convolutional Neural Networks (CNNs)

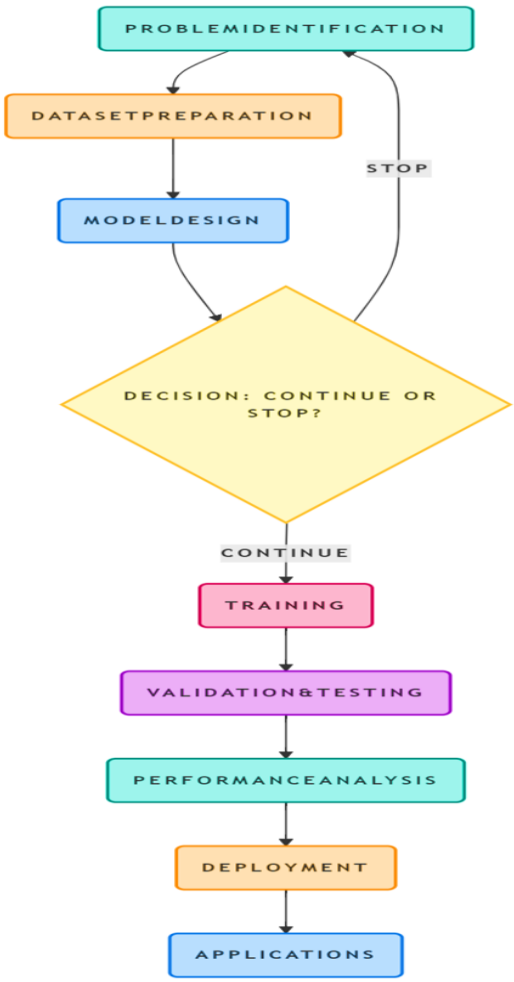
- CNNs are highly efficient for image recognition and feature extraction.
- **Key CNN components**:
  - **Convolution layers** → Detect patterns like edges, curves, and textures.
  - **Pooling layers** → Reduce dimensionality while preserving important features.
  - **Fully connected layers** → Classify the extracted features into digit categories.

b) Residual Networks (ResNet)

- ResNet introduces **skip connections** to solve the **vanishing gradient problem** in deep networks.
- **Key benefits of ResNet**:
  - Allows training of **ultra-deep networks** without performance degradation.
  - Learns intricate patterns, improving accuracy on **noisy and distorted data**.
  - Outperforms traditional CNNs in recognizing complex handwriting variations.

Dataset and Implementation

- **Dataset Used → MNIST (Modified National Institute of Standards and Technology)** dataset, containing **60,000 training** and **10,000 testing** images of handwritten digits.
- **Implementation Goals**:
  - Develop a **CNN-based model** for digit classification.
  - Implement **ResNet** to enhance feature extraction and accuracy.
  - Optimize the model for **high computational efficiency**.



Code:

a) Convolutional Neural Networks (CNNs) Model:

```
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt
import cv2
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten,Conv2D, MaxPooling2D
from tensorflow.keras.datasets import mnist
from google.colab import files
from PIL import Image
# Load MNIST dataset using Keras
(train_images, train_labels), (test_images, test_labels) =
mnist.load_data()

# Reshape and normalize the images
train_images = train_images.reshape(-1, 28, 28, 1) / 255.0
```

```
test_images = test_images.reshape(-1, 28, 28, 1) / 255.0

print(f"Training samples: {train_images.shape[0]}, Test samples:
{test_images.shape[0]}")
# Define the CNN Model
model = Sequential([
    Conv2D(32, kernel_size=(3,3), activation='relu',
input_shape=(28,28,1)),
    MaxPooling2D(pool_size=(2,2)),
    Conv2D(64, kernel_size=(3,3), activation='relu'),
    MaxPooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')  # Output layer for 10 digit
classes
])

# Compile the model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])

# Train the model
model.fit(train_images, train_labels, epochs=5,
validation_data=(test_images, test_labels))

test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test Accuracy: {test_acc * 100:.2f}%")

model.save("handwritten_digit_model.keras")

import tensorflow as tf # Import TensorFlow library

# Load the trained model
model = tf.keras.models.load_model("handwritten_digit_model.keras")

def preprocess_image(filename):
    img = Image.open(filename).convert("L")  # Convert to grayscale
    img = img.resize((28, 28))  # Resize to 28x28
    img = np.array(img) / 255.0  # Normalize
    img = img.reshape(1, 28, 28, 1)  # Reshape for model
    return img
# User uploads image
uploaded = files.upload()
for filename in uploaded.keys():
    img = preprocess_image(filename)

    plt.imshow(img.reshape(28,28), cmap='gray')
    plt.show()
```

```
prediction = model.predict(img)
predicted_digit = np.argmax(prediction)
print(f"Predicted Digit: {predicted_digit
```

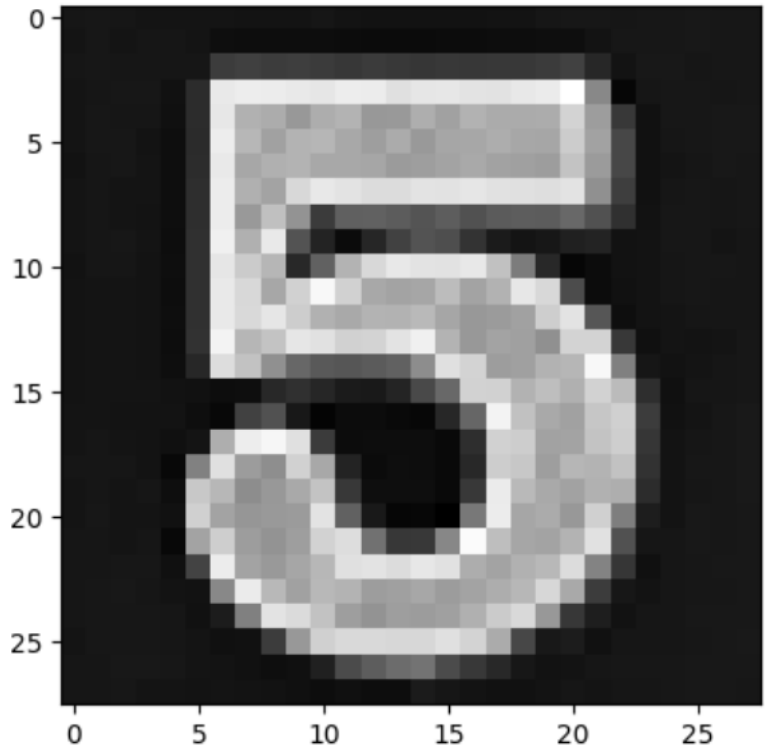
Output of Convolutional Neural Networks (CNNs) Model:

Commands + Code + Text RAM Disk

[7] Choose Files digit5.png

- **digit5.png**(image/png) - 1329886 bytes, last modified: 3/9/2025 - 100% done

```
Saving digit5.png to digit5.png
```



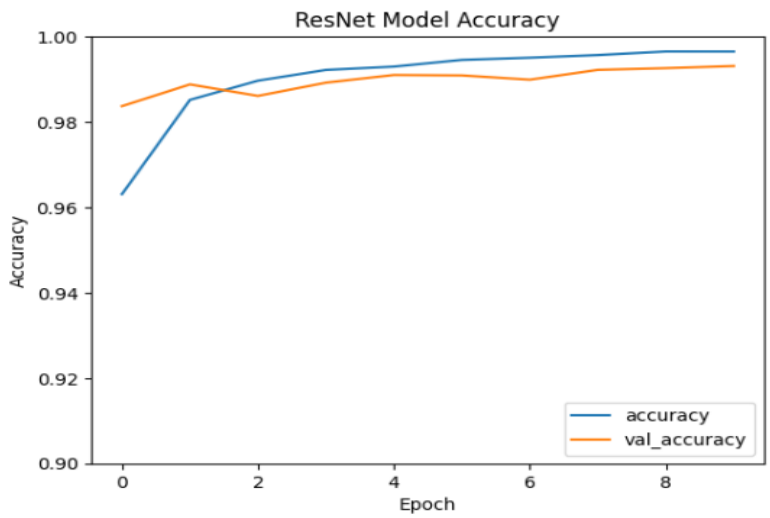

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 98ms/step
Predicted Digit: 5
```

b) Residual Networks (ResNet) Model:

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

# Load and preprocess the MNIST dataset
(x_train, y_train), (x_test, y_test) = datasets.mnist.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))

# Define the ResNet model
def resnet_block(inputs, num_filters, kernel_size=3, stride=1):
    x = layers.Conv2D(num_filters, kernel_size=kernel_size,
strides=stride, padding='same', activation='relu')(inputs)
    x = layers.BatchNormalization()(x)
    x = layers.Conv2D(num_filters, kernel_size=kernel_size, strides=1,
padding='same', activation='relu')(x)
    x = layers.BatchNormalization()(x)

    # If the input and output dimensions do not match, adjust them
    if stride != 1:
        inputs = layers.Conv2D(num_filters, kernel_size=1,
strides=stride, padding='same')(inputs)

    x = layers.add([x, inputs])
    x = layers.Activation('relu')(x)
    return x
inputs = layers.Input(shape=(28, 28, 1))
x = resnet_block(inputs, 32, stride=1)
x = layers.MaxPooling2D(pool_size=2, strides=2)(x)
x = resnet_block(x, 64, stride=2)
x = layers.MaxPooling2D(pool_size=2, strides=2)(x)
x = layers.Flatten()(x)
x = layers.Dense(128, activation='relu')(x)
outputs = layers.Dense(10, activation='softmax')(x)

model_resnet = models.Model(inputs, outputs)

# Compile the model
model_resnet.compile(optimizer='adam',
                     loss='sparse_categorical_crossentropy',
                     metrics=['accuracy'])
# Train the model
history_resnet = model_resnet.fit(x_train, y_train, epochs=10,
validation_data=(x_test, y_test))
```

```
# Evaluate the model
test_loss_resnet, test_acc_resnet = model_resnet.evaluate(x_test,
y_test, verbose=2)
print('\nTest accuracy (ResNet):', test_acc_resnet)

# Plot training and validation accuracy
plt.plot(history_resnet.history['accuracy'], label='accuracy')
plt.plot(history_resnet.history['val_accuracy'], label='val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.9, 1])
plt.legend(loc='lower right')
plt.title('ResNet Model Accuracy')
plt.show()

# Save the model
model_resnet.save('resnet_model.keras')

# Load the model and make predictions
model_loaded = tf.keras.models.load_model('resnet_model.keras')
predictions = model_loaded.predict(x_test)
```

```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 5ms/step - accuracy: 0.9205 - loss: 0.2781
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9851 - loss: 0.0449 - val_accuracy: 0.9888 - val_loss: 0.034
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9902 - loss: 0.0322 - val_accuracy: 0.9861 - val_loss: 0.04
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9921 - loss: 0.0239 - val_accuracy: 0.9892 - val_loss: 0.036
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9937 - loss: 0.0203 - val_accuracy: 0.9910 - val_loss: 0.03
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9947 - loss: 0.0164 - val_accuracy: 0.9909 - val_loss: 0.030
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9951 - loss: 0.0145 - val_accuracy: 0.9899 - val_loss: 0.043
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9960 - loss: 0.0132 - val_accuracy: 0.9922 - val_loss: 0.026
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9969 - loss: 0.0095 - val_accuracy: 0.9926 - val_loss: 0.02
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9967 - loss: 0.0101 - val_accuracy: 0.9931 - val_loss: 0.02
313/313 - 1s - 3ms/step - accuracy: 0.9931 - loss: 0.0290

Test accuracy (ResNet): 0.9930999875068665
```




```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import cv2

# Load the pre-trained ResNet model
model_resnet = tf.keras.models.load_model('resnet_model.keras')
# Assuming you saved it as .keras

# Function to preprocess user input
def preprocess_input(image):
    image = image / 255.0
    image = image.reshape((1, 28, 28, 1))
    return image

# Function to predict the digit
def predict_digit(image):
    processed_image = preprocess_input(image)
    prediction = model_resnet.predict(processed_image)  # Use the loaded
model
    predicted_digit = np.argmax(prediction)
    return predicted_digit

# Get user input
image_path = input("Enter the path to your image: ") # Get image path from
user
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)  # Read as grayscale
image = cv2.resize(image, (28, 28))  # Resize to 28x28

# Make a prediction
predicted_digit = predict_digit(image)

# Display the prediction
print('Predicted digit:', predicted_digit)

# Train the model
history_resnet = model_resnet.fit(x_train, y_train, epochs=10,
validation_data=(x_test, y_test))
```

Commands + Code + Text Reconnect T4

```
# Train the model
history_resnet = model_resnet.fit(x_train, y_train, epochs=10, validation_data=(x_test, y_test))
```

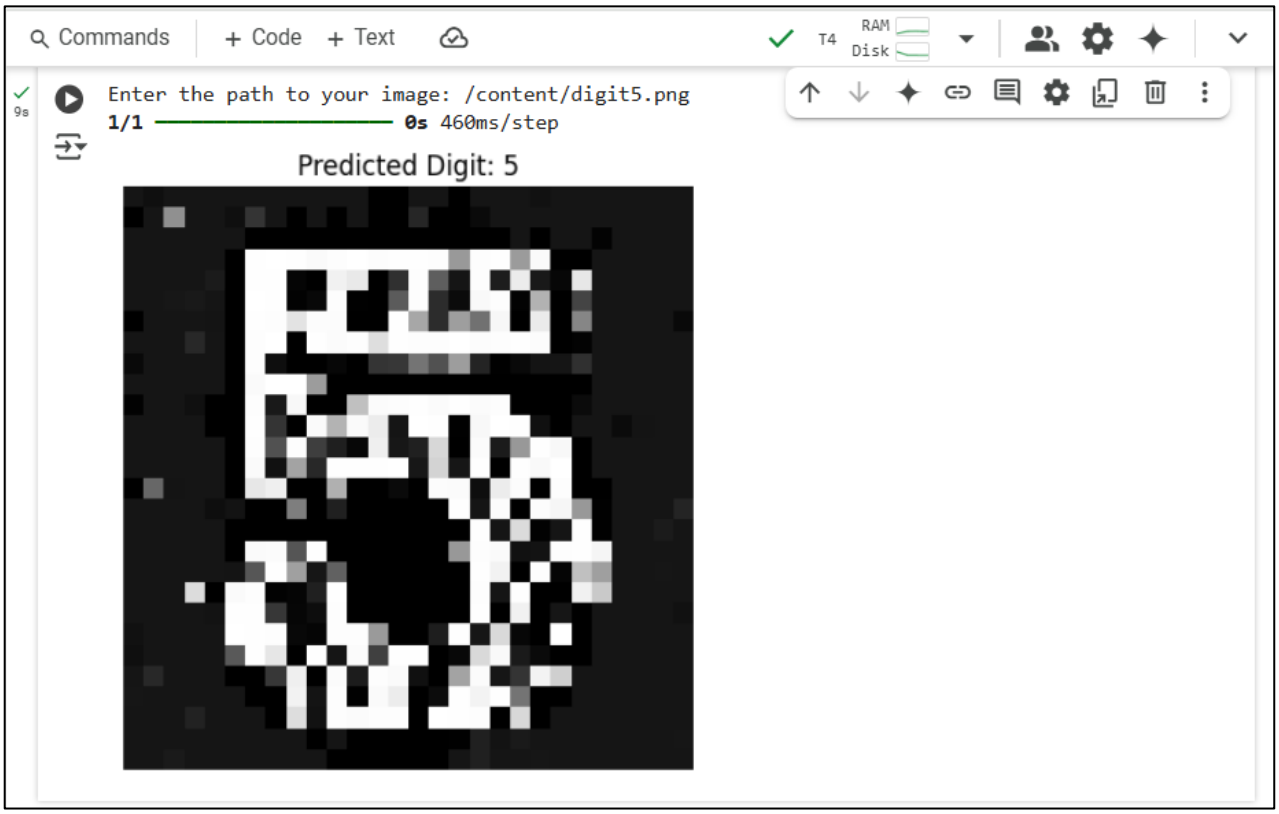
```
Epoch 1/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 15s 6ms/step - accuracy: 0.9972 - loss: 0.0087 - val_accuracy: 0.9911 - val_loss: 0.03
Epoch 2/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 16s 4ms/step - accuracy: 0.9979 - loss: 0.0067 - val_accuracy: 0.9921 - val_loss: 0.03
Epoch 3/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 11s 5ms/step - accuracy: 0.9980 - loss: 0.0059 - val_accuracy: 0.9906 - val_loss: 0.03
Epoch 4/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9976 - loss: 0.0070 - val_accuracy: 0.9915 - val_loss: 0.03
Epoch 5/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9974 - loss: 0.0084 - val_accuracy: 0.9916 - val_loss: 0.039
Epoch 6/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 5ms/step - accuracy: 0.9984 - loss: 0.0051 - val_accuracy: 0.9929 - val_loss: 0.03
Epoch 7/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 10s 4ms/step - accuracy: 0.9982 - loss: 0.0051 - val_accuracy: 0.9893 - val_loss: 0.04
Epoch 8/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 5ms/step - accuracy: 0.9983 - loss: 0.0062 - val_accuracy: 0.9922 - val_loss: 0.034
Epoch 9/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9991 - loss: 0.0031 - val_accuracy: 0.9927 - val_loss: 0.036
Epoch 10/10
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 8s 4ms/step - accuracy: 0.9987 - loss: 0.0037 - val_accuracy: 0.9917 - val_loss: 0.038
```

```
model_resnet.save('resnet_model.keras')
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import cv2

# Load the pre-trained ResNet model
model_resnet = tf.keras.models.load_model('resnet_model.keras')

# Function to preprocess user input
def preprocess_input(image):
    image = image / 255.0
    image = image.reshape((1, 28, 28, 1))
    return image

# Function to predict the digit
def predict_digit(image):
    processed_image = preprocess_input(image)
    prediction = model_resnet.predict(processed_image)
    predicted_digit = np.argmax(prediction)
    return predicted_digit

# Get user input
image_path = input("Enter the path to your image: ")
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
image = cv2.resize(image, (28, 28))

# Make a prediction
predicted_digit = predict_digit(image)
```

```
# Display the image and prediction
plt.imshow(image, cmap='gray')  # Display image in grayscale
plt.title(f"Predicted Digit: {predicted_digit}")
plt.axis('off')  # Hide axes
plt.show()
```

Output of Residual Networks (ResNet) Model:



Comparative Analysis:

Algorithm	Precision				Recall				F-1 Score				Accuracy
	0	1	Macro avg	Wt. Avg	0	1	Macro avg	Wt. Avg	0	1	Macro avg	Wt. Avg	
CNN Model	99%	99%	98.87%	98.87%	99%	99%	98.84%	98.86%	99%	99%	98.85%	98.86%	**98.86%**
ResNet Model	98%	99%	99.11%	99.12%	99%	99%	99.11%	99.12%	99%	99%	99.11%	99.12%	**99.12%**

From above 2, ResNet Model is working well on dataset.

Conclusion:

The project demonstrates the successful application of advanced deep learning techniques, utilizing both CNN and ResNet models, to develop an efficient and reliable handwritten digit recognition system. By tackling challenges like noise and handwriting variability, it ensures adaptability to real-world scenarios. The incorporation of ResNet enhances the system's ability to learn complex patterns, ensuring high performance even in the presence of challenging data.

Additionally, the comparative evaluation of models highlights the strengths of CNNs and ResNet, providing valuable insights into their effectiveness. With robust generalization achieved through diverse training, the project delivers a tool that bridges the gap between academic innovation and practical utility. Its user-centric design ensures accessibility and usability across various domains, making it a significant contribution to automation and digitization efforts.

Hence, we studied ResNet model and give 99.12 % accuracy ensure high accuracy and computational efficiency.

Evaluation Parameters

Sr. No.	USN No.	Name of Student	Faculty Assessment (4M)	Submission (3M)	Viva (3M)	Total (10M)	Signature
1.	AD22054	Divyanshi Bokade					
2.	AD22055	Vaibhav Charde					
3.	AD22056	Samarth Sharma					
4.	AD22057	Peeyush Kaware					

Signature of Course In-Charge